Backend: accounts, storage, enrichment and AI

The server under `server/` replaces the browser-only prototype (`auth.js`, `localStorage`) with a small, secured service. It serves the built pages and the API from one origin.

Run

```
npm install && npm run build
```

```
cd server && npm install && npm start
```

Open `http://127.0.0.1:8787/`. The first visit asks for the administrator account; add analysts, reviewers and viewers from the desk's **Users** panel afterwards.

Development with hot reload: run the server as above and `npm run dev` in the root; Vite proxies `/api` to the server.

ENVIRONMENT

Variable	Default	Meaning
`PORT`, `HOST`	`8787`, `127.0.0.1`	Listen address. Put a TLS-terminating reverse proxy in front for anything beyond localhost and set `CIB_TRUST_PROXY=1`.
`CIB_DATA_DIR`	`server/data`	SQLite database and the file store. Back this directory up.
`CIB_SECURE_COOKIES`	`1` when `NODE_ENV=production`	Mark the session cookie `Secure` (HTTPS only).
`CIB_SESSION_HOURS`, `CIB_IDLE_MINUTES`	`12`, `15`	Absolute and idle session limits (spec §10).
`CIB_LOCKOUT_FAILURES`, `CIB_LOCKOUT_MINUTES`	`10`, `15`	Failed sign-ins before an account locks, and for how long.
`CIB_MIN_PASSWORD`	`8`	Minimum password length (a common-password list and "not your name" are always enforced).
`CIB_MAX_FILE_MB`	`50`	Largest media file accepted by the file store.
`CIB_PROXY_ALLOW`	see `server/src/config.js`	Comma-separated hosts the enrichment proxy may reach.
`THREATFOX_AUTH_KEY`, `VT_API_KEY`, `SHODAN_API_KEY`, `GREYNOISE_API_KEY`, `ABUSEIPDB_API_KEY`, `URLSCAN_API_KEY`	unset	Provider keys the proxy attaches server-side. A host is added to the proxy allow-list only when its key is set; the browser never sees the key. abuse.ch (ThreatFox, URLhaus, MalwareBazaar) now requires an `Auth-Key` even for free use.
AI provider keys (see **AI providers** below)	unset	Any number of model providers; they are tried in order and the built-in engine answers when none is configured or all fail.

What is secured, and how

- **Passwords**: scrypt (N=2^15, r=8, p=1, 64-byte key, 16-byte salt), constant-time comparison. Minimum length plus a common-password list, "not one repeated character" and "not containing your name". Unknown user names take the same time to reject as wrong passwords.
- **Sessions**: 256-bit random token, stored hashed; cookie `cib_session` is `HttpOnly`, `SameSite=Strict`, `Secure` in production, 12 h absolute and 15 min idle expiry. Logout and admin password resets invalidate sessions.
- **Brute force**: per-route rate limits (20 sign-ins per 15 min per IP, 10 setups, 600 API calls per minute) and a per-account lock after 10 failures for 15 minutes. Every failure is in the audit log.
- **CSRF**: `SameSite=Strict` cookie, a required `X-Requested-With: CIB` header on every write, and an `Origin`/`Host` match check.
- **Headers**: Helmet with a content-security policy (`script-src 'self'`; images, media and connections to `https:` for OSINT lookups, maps and embeds; frames only for known players), `frame-ancestors 'none'`, no referrer, HSTS when cookies are secure.
- **Authorisation** (BACKEND-SPEC §10): the same four checks the pages use, evaluated server-side on every request and on every live event: `canSee` (role, per-user case list, TLP:RED / AMBER+STRICT grant, public + CLEAR/GREEN for guests), `canEdit` (admin, analyst), `canReview`

(reviewers may only change `review`, `signoff`, `archived`, `desk`, `color`, `tags`, `log`), `canCreate`, `isAdmin`. Hidden cases return 404, not 403.

- **Audit**: append-only table where each row hashes the previous one; `/api/health` reports whether the chain verifies. Sign-ins, failures, lockouts, user changes, resets, case creation, deletion and stale overwrites are recorded with the client IP.
- **Files**: content-addressed (SHA-256) immutable store under `CIB_DATA_DIR/files`, written once, served only to signed-in accounts with the hash in a response header. Sizes are capped.
- **Enrichment proxy**: HTTPS only, explicit host allow-list, no credentials in URLs, DNS resolution checked against private ranges before each request (SSRF guard), 12 s timeout, 4 MB cap, rate-limited, accounts only. Replaces the third-party `allorigins.win` proxy the design used.
- **AI gateway**: the browser never holds a model key. Prompts go to the server, which calls whichever providers are configured, in order, and falls back to the built-in engine.

NOT DONE HERE, AND WORTH KNOWING

- TLS, backups and an identity provider (OIDC) are deployment concerns; the spec's OIDC flow can replace `/api/auth/*` without touching the pages.
- Media files are stored per upload, not per case, so file reads are allowed to any signed-in account; TLP enforcement applies to cases, not to individual files.
- Client-side encryption at rest (the board's **Encrypt** button) still works locally but an encrypted browser store cannot be synced; the server is the system of record.

API

All routes are under `/api`. Writes need the `X-Requested-With: CIB` header. Errors are `{ "error": "message" }`.

Method and path	Who	Purpose
`GET /health`	anyone	Liveness, user count, audit chain status, AI engine.
`GET /auth/session`	anyone	Current user, whether setup is done, whether public files exist.
`POST /auth/setup`	nobody yet	First-run: create the admin (409 afterwards).
`POST /auth/login`, `POST /auth/logout`, `POST /auth/touch`	—	Sign in (sets the cookie), sign out, refresh idle timer.
`POST /auth/guest`	anyone	Read-only guest session, only while a public CLEAR/GREEN case exists.
`POST /auth/password`	account	Change own password (clears "must change").
`GET /users`, `POST /users`, `PATCH /users/:id`, `POST /users/:id/password`, `DELETE /users/:id`	admin	Manage accounts, roles, TLP:RED grant and case scope; reset passwords (user must change on next sign-in).
`GET /store`	signed in	Visible cases (with `rev`), the user's desk and settings documents.
`GET /store/case/:id`	signed in	One case (404 when hidden).
`PUT /store`	account	`{cases:[…], deleted:[ids], desk, settings}`. Returns accepted (`id`, `rev`, server `number`) and rejected (`id`, `reason`). Identical writes do not bump revisions; stale writes are applied last-writer-wins and audited.
`PUT /files/:id`, `GET /files/:id`	editors / accounts	Media mirror; body is the raw file, response carries `sha256`.
`GET|POST /proxy?url=`	account	Fetch an allow-listed HTTPS URL on the browser's behalf.
`POST /ai/complete`	account	`{system, messages, max_tokens}` → `{text, engine}`.
`GET /events`	signed in	Server-sent events: `case` (`id`, `rev`, `by`) and `case-deleted`, filtered by what the listener may see.

How the pages use it

`src/runtime/` holds the glue, none of it in the generated page code:

- `auth.js` exposes `window.CIBAuth` with the same methods the design's `auth.js` had. Synchronous reads come from a cache filled by `init()` before the page renders; writes call the API and update the cache optimistically.
- `store.js` pulls the user's cases into the pages' `localStorage` documents before render, intercepts writes to those keys, pushes changed cases (debounced, per case, with revisions) and applies other people's changes from the event stream unless the same case has unsaved local edits.
- `claude.js` makes `window.claude.complete` call the gateway. `proxy.js` sends CORS-blocked hosts through `/api/proxy`. Media `putBlob`/`getBlob` are mirrored to `/api/files`.

AI providers

The gateway behind `POST /api/ai/complete` (and therefore behind "Analyse with Claude", translate, stance and entities) can use as many model providers as you give it keys for. They are tried in order; a provider that fails (bad key, quota, timeout, empty answer) is skipped and the next one answers, with the failure recorded in the audit line and returned to the client as `tried`. A provider that failed is rested for `AI_COOLDOWN_S` (default 120 s) unless the failure was a 429. Identical prompts are served from a short cache (`AI_CACHE_TTL_S`, default 600 s). The built-in rule engine is always last, so the buttons never dead-end.

Provider	**Configure**	**Default model (override with the `_MODEL` variable)**
Anthropic	`ANTHROPIC_API_KEY`	`ANTHROPIC_MODEL` = `claude-sonnet-5-5`
OpenAI	`OPENAI_API_KEY` (`OPENAI_BASE_URL` optional)	`OPENAI_MODEL` = `gpt-4.1-mini`
Azure OpenAI	`AZURE_OPENAI_API_KEY`, `AZURE_OPENAI_ENDPOINT`, `AZURE_OPENAI_DEPLOYMENT` (`AZURE_OPENAI_API_VERSION`)	the deployment
Google Gemini	`GEMINI_API_KEY` or `GOOGLE_API_KEY`	`GEMINI_MODEL` = `gemini-2.5-flash`
Cohere	`COHERE_API_KEY`	`COHERE_MODEL` = `command-a-03-2025`
Mistral	`MISTRAL_API_KEY`	`MISTRAL_MODEL` = `mistral-small-latest`
Groq	`GROQ_API_KEY`	`GROQ_MODEL` = `llama-3.3-70b-versatile`
OpenRouter (any model behind it)	`OPENROUTER_API_KEY` (`OPENROUTER_SITE`)	`OPENROUTER_MODEL` = `anthropic/claude-sonnet-4`
DeepSeek	`DEEPSEEK_API_KEY`	`DEEPSEEK_MODEL` = `deepseek-chat`
xAI	`XAI_API_KEY`	`XAI_MODEL` = `grok-3-mini`
Together	`TOGETHER_API_KEY`	`TOGETHER_MODEL` = `meta-llama/Llama-3.3-70B-Instruct-Turbo`
Perplexity	`PERPLEXITY_API_KEY`	`PERPLEXITY_MODEL` = `sonar`
Fireworks	`FIREWORKS_API_KEY`	`FIREWORKS_MODEL` = `accounts/fireworks/models/llama-v3p3-70b-instruct`

Provider	Configure	Default model (override with the `_MODEL` variable)
Ollama (local, no key)	`OLLAMA_MODEL` (`OLLAMA_BASE_URL` = `http://127.0.0.1:11434`)	the model you pulled
LM Studio (local, no key)	`LMSTUDIO_BASE_URL` (`LMSTUDIO_MODEL`)	loaded model
Any OpenAI-compatible endpoint	`AI_CUSTOM_BASE_URL`, `AI_CUSTOM_API_KEY`, `AI_CUSTOM_MODEL`, `AI_CUSTOM_NAME`	as given

Order: `AI_PROVIDERS=gemini,anthropic,ollama` puts those first (others follow unless `AI_PROVIDERS_STRICT=1`); `AI_DISABLED=openai` removes one; `AI_TIMEOUT_MS` (default 60000) bounds each call. Model names change often; the defaults are a starting point, not a guarantee that a given account has access to them.

Endpoints: `GET /api/ai/providers` lists the chain (admins also see last status, error and latency); `POST /api/ai/providers/:id/test` (admin) sends a one-word health check through one provider; `PATCH /api/ai/providers/:id` with `{"disabled": true|false}` toggles it at runtime. A request may pin a provider with `"provider": "gemini"`. In the browser `window.claude.preferred = 'gemini'` pins it for that tab, and a toast says which engine answered whenever the gateway fell back.

Keys never leave the server; the pages only ever call `/api/ai/complete`.

The built-in analyst engine

Without a model key the gateway still answers the board's four prompts, from rules, and says so in its text:

- **Assess**: parses the prompt the board sends (card, other cards, threads, notes), stamps from the card's status or source, suggests threads to cards that share identifiers (relation chosen from the card types), and proposes next steps per indicator kind.
- **Entities**: regex and pattern extraction for emails, phones, money, dates, CVEs, IPs, hashes, organisations, people, places, vehicles and plates.
- **Stance**: cue-word sentiment and support/oppose phrases, with the most repeated capitalised phrase as the narrative.
- **Translate**: language detection by script and stop-words; the text is returned unchanged with a note, since no translation model is available offline.

Configure any provider above to replace all four with a model.

Tests

```
cd server && npm test
```

Covers setup, CSRF and origin checks, weak passwords, roles and scopes, lockout and reset, server-assigned numbers and revisions, reviewer and viewer limits, guests, files, proxy guards, the local AI shapes, logout and the audit chain. The end-to-end browser run is `node tools/capture/v2.cjs` (see `docs/V2.md`).